Department Of electrical and computer Engineering

ADVANCED DIGITAL SYSTEMS DESIGN

ENCS3310

Project report

Instructor: Dr. Abdallatif AbuIssa

Made By: yara khattab

ID: 1210520

Section 1

Date:21-1-2024

Abstract:

This project involves the creation of a simple microprocessor with two main components, an Arithmetic Logic Unit (ALU) and a Register File .The ALU supports operations, including arithmetic and logical operations and the Register File operating as a small, fast RAM with 32 x 32-bit words. The microprocessor processes 32-bit machine code instructions, including opcode, source registers, and destination register fields.

Table of Contents:

Table of Figures:

Table of tables:

Part 1

1) The ALU

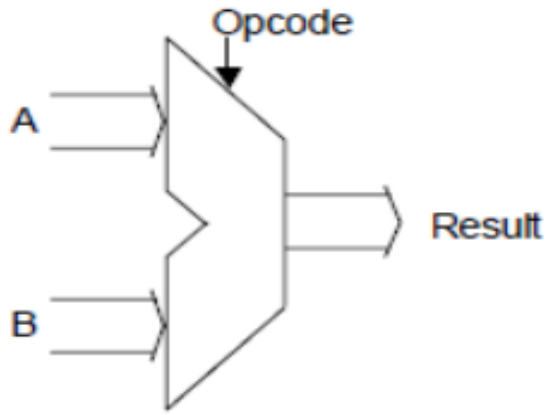

Figure 1:ALU block digram

The ALU has two 32-bit inputs A and B , a 32-bit output Result and a 6-bit for opcode.

According to the value of the opcode, The result is derived from one or both of the inputs.

The ALU operations:

- a + b ➔ (Addition)
- a – b ➔ (Subtraction)
- |a| ➔ (Absolute Value)
- -a ➔(Negtive)
- max (a, b) ➔ (Max Value)
- min (a, b) ➔ (Min Value)
- avg(a,b) ➔ (Avg Value)
- not a ➔(Negation)
- a or b ➔(Oring)
- a and b ➔(Anding)
- a xor b ➔(XOR)

The opcode that will be used to represent each of the above operations is determined by the last digit of my ID number. ➔**my ID** 1210520 **.**

Digit of ID no.	0	1	2	3	4	5	6	7	8	9
a + b	8	6	6	4	6	3	4	4	1	5
a – b	9	8	9	11	2	15	10	14	6	8
|a|	2	10	1	8	5	13	3	8	13	13
-a	10	12	5	6	4	12	12	11	8	7
max(a,b)	12	14	7	13	7	7	7	10	7	3
min (a,b)	1	11	8	14	10	1	2	1	4	6
avg(a,b)	13	13	11	7	9	9	6	13	11	10
not a	5	15	14	9	13	10	13	6	15	2
a or b	4	2	13	12	8	14	14	9	3	15
a and b	11	3	12	10	1	11	11	5	5	4
a xor b	15	9	4	5	3	5	8	7	2	12

Table 1: opcodes

➔change the digits for opcode to hexadecimal or binary

a + b	8	➔001000
a – b	9	➔001001
|a|	2	➔000010
-a	10	➔001010
max(a,b)	12	➔001100
min (a,b)	1	➔000001
avg(a,b)	13	➔001101
not a	5	➔000101
a or b	4	➔000100
a and b	11	➔001011
a xor b	15	➔001111

Table 2: opcodes in binary

The code for ALU:

```
module alu (opcode, a, b, result );
input  [5:0] opcode;
input signed [31:0] a, b;
output reg [31:0] result;
always@(*)
    begin
        case(opcode)
            6'b001000 : result = a+b;//ADD
            6'b001001 : result = a-b;//SUB
            6'b000010 :begin
                       case (a < 0)
                          1: result =-a;
                          0: result = a;//ABS
                       endcase
                       end
            6'b001010 : result = -a;
            6'b001100 :begin
                         if (a > b)
                            result=a ;//MAX
                         else
                            result= b;
                        end
            6'b000001 :begin
                         if (a < b)
                            result =a ; //MIN
                         else
                            result= b;
                         end
            6'b001101 :begin
                       result =(a+b)/(2) ;//AVG
                     end
            6'b000101 : result = ~(a);//NOT
            6'b000100 : result = a | b ; //OR
            6'b001011 : result = a & b;  //AND
            6'b001111 : result = a ^ b;  //XOR
            default: result = 0;
        endcase
    end
endmodule
```

Figure 2:ALU code

This code have three inputs a,b ,opcode and one output result .and it has a 11 different opcodes with different operations ,so the output result depends on the value of the opcode and the values of a and b , as shown in the figure 2 above .

Alu code works correctly for each operation.

2) The Register File

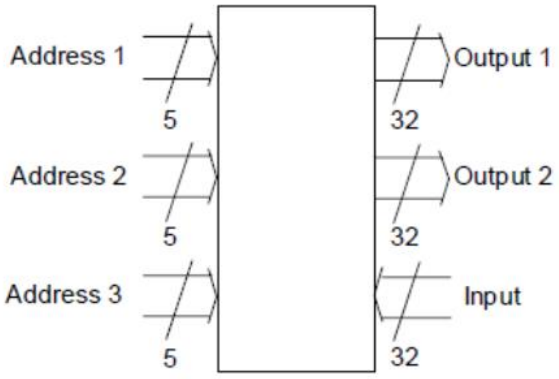

Figure 3:Register File block digram

This is a very small fast RAM is designed for access to 32 x 32-bit words, using a 5-bit address. It has three addresses : Output 1 reads from Address 1, Output 2 reads from Address 2, and Input writes to Address 3.

The initial values stored in the register file are determined by the second-from-last digit of my student ID ➔1210520 , shown in the table below:

ID/ Location	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	7942	11662	12642	12996	4198	11930	4616	15034	5986	16302
2	13224	11562	10592	11490	5596	5348	11640	8854	12250	2994
3	15462	15330	6230	7070	14426	7308	11254	170	482	1658
4	8026	9594	8940	6026	7612	15684	6786	7226	14246	5474
5	3692	14746	8776	3322	6638	12346	6784	4480	5124	6784
6	9882	3288	9436	10344	10040	9716	12432	8928	1848	10836
7	8248	5932	3056	6734	3930	7820	13548	7302	5260	4648
8	3432	1978	4850	15834	4150	5190	13462	8922	16170	524
9	178	4912	3406	15314	6406	14702	13454	1044	4766	12200
10	2378	2380	12380	6000	5400	5630	11780	6706	4298	3286
11	8302	1926	548	12196	8572	2352	13170	258	610	14734
12	592	12726	13054	11290	16324	15424	2982	7354	1510	10998
13	7430	176	2800	13350	8840	2670	8096	3294	9794	4420
14	10572	8408	12988	2086	8258	4172	514	14740	7456	8754
15	7676	8394	956	6734	11228	4300	3600	6532	5580	3246
16	1238	13604	2194	7430	8462	4744	10870	10436	9300	9040
17	16008	10222	11914	14102	13284	1286	12528	11900	12314	8714
18	2426	7262	15864	13200	4676	8122	9860	14694	12806	12008
19	11930	10190	11832	3264	3980	4558	6166	8830	10478	1006
20	6724	8734	12346	2368	5634	8534	4520	8712	11556	6724
21	12790	12432	2192	15846	7632	13340	14436	4532	6778	12746
22	4842	8724	1840	11710	9846	6918	12136	9084	8430	5462
23	7108	5412	13996	14736	5442	11700	5134	13838	5700	11810
24	6296	11082	12054	5338	12488	10722	11958	10018	13422	7590
25	3322	2212	4434	5544	6656	3346	7688	1280	11224	4368
26	10848	6188	12152	1852	832	3300	5258	5814	1990	10358
27	14698	7056	9876	3898	4664	2386	12420	670	922	15252
28	16378	3744	8734	16252	6798	11212	3560	8832	6020	11954
29	15456	5766	8308	1048	14166	3504	1248	15186	15768	13704
30	1260	3412	3422	5642	3246	8712	8724	4512	5624	1478
31	0	0	0	0	0	0	0	0	0	0

Table 3:Register File table

The code for register file:

```
module regfile (clk, valid_opcode, addr1, addr2, addr3, in , out1, out2);
input clk;
input valid_opcode;
input [4:0] addr1, addr2, addr3;
input [31:0] in;
output reg [31:0] out1, out2;
reg [31:0] MyMeMory [0:31]= '{32'h0 ,32'h3162,32'h2960,32'h1856,32'h22EC,32'h2248,32'h24DC,32'hBF0,32'h12F2,32'hD4E,32'h305C,32'h224,32'h32FE,32'hAF0,
                        32'h32BC,32'h3BC,32'h892,32'h2E8A,32'h3DF8,32'h3E38,32'h303A,32'h890,32'h730,32'h36AC,32'h2F16,32'h1152,32'h2F78,32'h2694,
                        32'h221E,32'h2074,32'hD5E,32'h0};

  always @ (posedge clk)
      begin

      if ( valid_opcode)
        begin
          out1 <= MyMeMory[addr1];//read
          out2 <= MyMeMory[addr2];//read
          MyMeMory[addr3] <= in; //write
    end
    end


endmodule
```

Figure 4: Register File code

In this code I created an array memory named MyMeMory and filled it as required, also I convert the data from decimal to hexadecimal before filled them.

Clk ➔works only on rising edge of the clock.

RegFile module works correctly.

Part 2

In this part I will connect the ALU with the RAM to form a simple microprocessor.

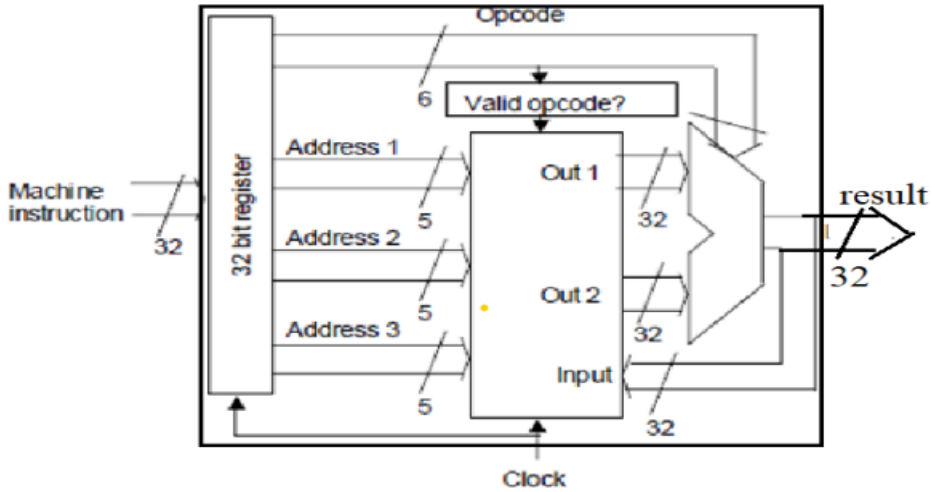

Figure 5:microprocessor block digram

The instructions for this system are 32-bit numbers organized as follows:

1)The initial 6 bits(from LSB) for the opcode

2)The next 5 bits for the first source register.

3)The next 5 bits for the second source register.

4)The next 5 bits for the destination register.

5)The remaining 11 bits are unused.

The code for simple microprocessor :

```
//module for dff
module DFF (clk,D1,Q1);
  input  clk;
  input [5:0] D1;
  output reg [5:0] Q1;
  always @(posedge clk ) begin

      Q1 = D1;
  end
endmodule

// module for Microprocessor
module mp_top(clk, instruction, result);
  input clk;
  input [31:0] instruction;
  output reg [31:0] result;
  // division fields for the instruction starting from LSB
  wire [5:0] opcode = instruction[5:0];
  wire [4:0] readfirstvalue = instruction[10:6];
  wire [4:0] readsecondvalue = instruction[15:11];
  wire [4:0] writetheresult = instruction[20:16];
  wire [10:0] unused = instruction[31:21];

  wire valid_opcode = (opcode != 6'b000000); // valid opcode
  //here i add a delay for the opcode by using dff
  reg [5:0]opcode2;
  DFF mydff(.clk(clk),.D1(opcode),.Q1(opcode2));

  // get instance from regfile and alu and connect them
  regfile Myregfile(.clk(clk), .valid_opcode(valid_opcode), .addr1(readfirstvalue), .addr2(readsecondvalue), .addr3(writetheresult), .in(0), .out1(), .out2());
  alu Myalu(.opcode(opcode2), .a(Myregfile.out1), .b(Myregfile.out2), .result(result));
  // unused bits
  assign unused = 11'b0;
endmodule

```

Figure 6:microprocessor (mp-top) code

In this code you can see that there two modules the first module is a simple module for DFF which has two inputs clk,D1 and one output Q1, the value in D1 is assigned to Q1 at every posedge of the clk(**only works on rising edge of the clock**) .

And the second module is for a simple microprocessor and it have two inputs clk(**only rising edge of the clock**),instruction(32 bits) and one output result (32 bits) , as also shown in the code I split the instruction field as required for the opcode, two source registers and a destination register , as well as 11 unused bits. And I used the DFF in this code to get delay with one clock cycle for the opcode ,because the opcode reaches the ALU before the data from registers so by this we can avoid errors in performing the instructions.

Mp-top module works correctly.

The test bench 1 code for mp-top :

```
module MyTestBench1;
  reg clk;
  reg [31:0] instruction;
  wire [31:0] result;
  // get instance of  mp_top module and connect it
  mp_top  MyMpTop(.clk(clk),.instruction(instruction),.result(result));

  // creat an array of instructions
  reg [31:0] instructions [0:12];
  integer i;//varible uses in for loop to excut the instructions
  integer count=0;//varible uses to count the number of tests fails
  initial begin
    clk = 0;//clock generation
    // here the instructions for each opcode to test
    instructions[1] = 32'b00000000000000001000100011001000; // ADD (R1=R3+R2)
    instructions[2] = 32'b00000000000000001000100011001001; // SUB (R1=R3-R2)
    instructions[3] = 32'b00000000000000001000100011000010; // ABS (R1=|R3|)
    instructions[4] = 32'b00000000000000001000100011001010; // NEG (R1= -R3)
    instructions[5] = 32'b00000000000000001000100011001100; // MAX (R1=max(R3,R2))
    instructions[6]= 32'b00000000000000001000100011000001; // MIN (R1= min(R3,R2))
    instructions[7]= 32'b00000000000000001000100011001101; // AVG (R1= avg(R3,R2))
    instructions[8]= 32'b00000000000000001000100011000101; // NOT (R1= ~R3)
    instructions[9]= 32'b00000000000000001000100011000100; // OR  (R1=R3 or R2)
    instructions[10]= 32'b00000000000000001000100011001011; // AND (R1=R3 and R2)
    instructions[11]= 32'b00000000000000001000100011001111; // XOR (R1=R3 xor R2)
    for (i = 0; i < 12; i = i + 1) begin
      instruction = instructions[i];
      #10ns; // Wait for a few clock cycles
      #5 clk = ~clk;
      //case stetment to compare the result with the expected value
    case(instruction)
        32'h000110c8: if(result!=32'h000041b6)begin //if statment to compare  opcode (a+b)
            count= count+1;//add 1 to count
            end
        32'h000110c9 : if(result!=32'hffffeef6) begin//if statment to compare  opcode(a-b)
            count= count+1;
            end
         32'h000110c2: if(result!=32'h00001856)begin//if statment to compare  opcode(|a|)
            count= count+1;
            end
```

Figure 7: test bench 1 of mp-top

```
41          32'h000110ca: if(result!=32'hfffffe7aa)begin//if statment to compare opcode(-a)
42             count= count+1;
43             end
44          32'h000110cc: if(result!=32'h00002960)begin//if statment to compare opcode(max)
45             count= count+1;
46          end
47         32'h000110c1: if(result!=32'h00001856)begin //if statment to compare  opcode(min)
48             count= count+1;
49        end
50        32'h000110cd: if(result!=32'h000020db)begin //if statment to compare  opcode(avg)
51
52             count= count+1;
53        end
54        32'h000110c5: if(result!=32'hfffffe7a9)begin//if statment to compare  opcode(not)
55             count= count+1;
56          end
57          32'h000110c4: if(result!=32'h00003976)begin//if statment to compare  opcode(or)
58             count= count+1;
59          end
60          32'h000110cb: if(result!=32'h00000840)begin//if statment to compare  opcode(and)
61             count= count+1;
62          end
63          32'h000110cf: if(result!=32'h00003136)begin //if statment to compare  opcode(xor)
64             count= count+1;
65          end
66     endcase
67      // display results
68       $display("Test Number %0d - Instruction Format: %h,  Result: %h", i, instruction, result);
69       end
70
71     // if statment to check if the count > 0
72     if(count > 0) begin
73     $display("Test fail in %0d tests ",count);// if count >0 then print test faild
74          end
75   else
76       begin
77       $display("All Tests passed");//if count still 0 then print all the tasts passed
78       end
79   end
80 endmodule
```

Figure 8: continue test bench 1 of mp-top

This is the test bench code of mp-top ,in this code I create an array of instructions to test all cases ,we have a 11 different opcode so I add a 11 instructions to test the opcodes ,testing instructions will be through a for loop which takes each instruction and evaluate the result then takes the next one until the instructions finished ,also in the code I used a case statement to compare the result and expected value of each given instruction so if the result dose not equal the expected value then increment the variable count(variable to count the number of failed tests) by 1 ,and at the end of the code I used if statemante to compare the variable count with zero so if count grater then zero this means there a failed test but if the count still equal zero this means that all the tests passed .

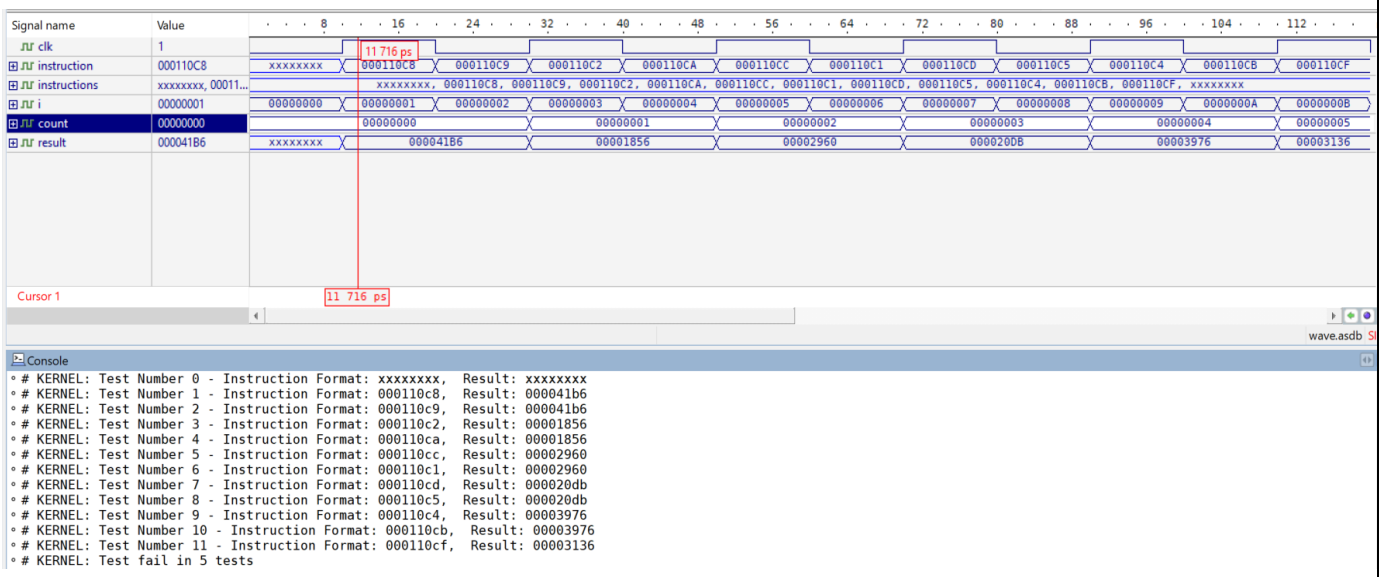

Figure 9: simulation of test bench 1 of mp-top

This is the simulation for the previous test bench 1 code ,from the simulation we can see that the first result (don't cares) this because we do not know the values stored at the first and also we can see that the result only changed when the rising egde of clock is positive so this explains why there is 5 tests fails (the 5 tests:SUB ,NEG,MIN,NOT,AND) theses five tests works in negative edge of the clock and display their results on negative edge but our project works only on positive edge ,therefor it will keep the current result until reaches to a new positive edge and change the result depends on current instruction and so on .

All the instructions have :

variable	Which register	In decimal	In hexadecimal
a	R3	6230	1856
b	R2	10592	2960
result	Store in R1		

Calculate The expected values of the instructions in decimal and hexadecimal :

Instruction	Expected value in decimal	Expected value in Hexa
R1=R3+R2	16822	32'h000041b6
R1=R3-R2	-4362	32'hffffeef6
R1= |R3|	6230	32'h00001856
R1= -R3	-6230	32'hffffe7aa
R1=max(R3,R2)	10592	32'h00002960
R1=min(R3,R2)	6230	32'h00001856
R1=avg(R3,R2)	8411	32'h000020db
R1= not(R3)	4294961065	32'hffffe7a9
R1= or(R3,R2)	14710	32'h00003976
R1= and(R3,R2)	2112	32'h00000840
R1= xor(R3,R2)	12598	32'h00003136

Table 4:calculate the expected value

```
module MyTestBench2;
  reg clk;
  reg [31:0] instruction;
  wire [31:0] result;
  // get instance of  mp_top module and connect it
  mp_top  MyMpTop(.clk(clk),.instruction(instruction),.result(result));

  // creat an array of instructions
  reg [31:0] instructions [0:23];
  integer i;//varible uses in for loop to excut the instructions
  integer count=0;//varible uses to count the number of tests fails
  initial begin
    clk = 0;//clock generation
    // here the instructions for each opcode to test
    instructions[1] = 32'b00000000000000001000100011001000; // ADD (R1=R3+R2)
    instructions[2] = 32'b00000000000000001000100011001000; // instruction add to test all cases
    instructions[3] = 32'b00000000000000001000100011001001; // SUB (R1=R3-R2)
    instructions[4] = 32'b00000000000000001000100011001001; // instruction add to test all cases
    instructions[5] = 32'b00000000000000001000100011000010; // ABS (R1=|R3|)
    instructions[6] = 32'b00000000000000001000100011000010; // instruction add to test all cases
    instructions[7] = 32'b00000000000000001000100011001010; // NEG (R1= -R3)
    instructions[8] = 32'b00000000000000001000100011001010; // instruction add to test all cases
    instructions[9] = 32'b00000000000000001000100011001100; // MAX (R1=max(R3,R2))
    instructions[10]= 32'b00000000000000001000100011001100; // instruction add to test all cases
    instructions[11]= 32'b00000000000000001000100011000001; // MIN (R1= min(R3,R2))
    instructions[12]= 32'b00000000000000001000100011000001; // instruction add to test all cases
    instructions[13]= 32'b00000000000000001000100011001101; // AVG (R1= avg(R3,R2))
    instructions[14]= 32'b00000000000000001000100011001101; // instruction add to test all cases
    instructions[15]= 32'b00000000000000001000100011000101; // NOT (R1= ~R3)
    instructions[16]= 32'b00000000000000001000100011000101; // instruction add to test all cases
    instructions[17]= 32'b00000000000000001000100011000100; // OR  (R1=R3 or R2)
    instructions[18]= 32'b00000000000000001000100011000100; // instruction add to test all cases
    instructions[19]= 32'b00000000000000001000100011001011; // AND (R1=R3 and R2)
    instructions[20]= 32'b00000000000000001000100011001011; // instruction add to test all cases
    instructions[21]= 32'b00000000000000001000100011001111; // XOR (R1=R3 xor R2)
    instructions[22]= 32'b00000000000000001000100011001111; // instruction add to test all cases
    // loop starts to excut the instructions
    for (i = 0; i < 23; i = i + 1) begin
      instruction = instructions[i];
      #10ns; // Wait for a few clock cycles
      #5 clk = ~clk;
      //case stetment to compare the result with the expected value
    case(instruction)
        32'h000110c8: if(result!=32'h000041b6)begin //if statment to compare  opcode (a+b)
            count= count+1;//add 1 to count
            end
        32'h000110c9 : if(result!=32'hffffeef6) begin//if statment to compare  opcode(a-b)
            count= count+1;
            end
         32'h000110c2: if(result!=32'h00001856)begin//if statment to compare  opcode(|a|)
            count= count+1;
            end
         32'h000110ca: if(result!=32'hffffe7aa)begin//if statment to compare opcode(-a)
            count= count+1;
            end
         32'h000110cc: if(result!=32'h00002960)begin//if statment to compare opcode(max)
            count= count+1;
         end
        32'h000110c1: if(result!=32'h00001856)begin //if statment to compare  opcode(min)
            count= count+1;
        end
        32'h000110cd: if(result!=32'h000020db)begin //if statment to compare  opcode(avg)

            count= count+1;
        end
        32'h000110c5: if(result!=32'hffffe7a9)begin//if statment to compare  opcode(not)
            count= count+1;
         end
         32'h000110c4: if(result!=32'h00003976)begin//if statment to compare  opcode(or)
            count= count+1;
         end
         32'h000110cb: if(result!=32'h00000840)begin//if statment to compare  opcode(and)
            count= count+1;
         end
         32'h000110cf: if(result!=32'h00003136)begin //if statment to compare  opcode(xor)
            count= count+1;
         end

    endcase

     // display results
      $display("Test Number %0d - Instruction Format: %h,  Result: %h", i, instruction, result);
      end

    // if statment to check if the count > 0
    if(count > 0) begin
    $display("Test fail in %0d tests ",count);// if count >0 then print test faild
        end
  else
      begin

      $display("All Tests passed");//if count still 0 then print all the tasts passed
      end


  end
endmodule
```

Figure 10: test bench 2 of mp-top

The previous code is the same code of test bench 1 for mp-top but I add some instructions to test all the operations on the ALU and show their results in the simulation .

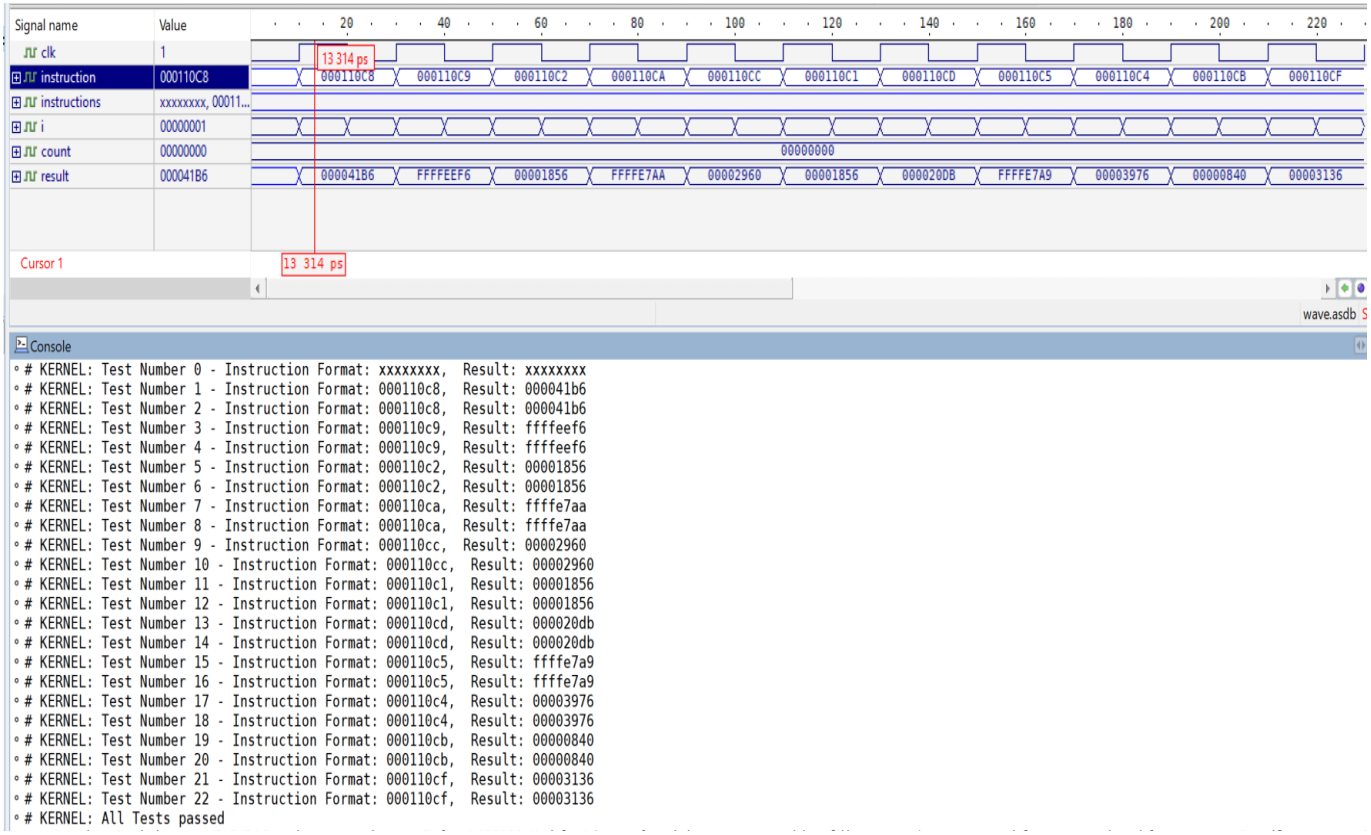

Figure 11: simulation of test bench 2 of mp-top

This is the mp-top simulation after adding some instructions to test all cases. We see from the simulation that all the tests succeeded, and we can see the result that is equal to the expected value shown in Table 4, and from this code and its simulation we can see the result of the five operations that did not appear in the test bench 1 (SUB,NEG,MIN,NOT,AND). Because it works on the negative edge of the clock .

Conclusion :

In the end , the simple microprocessor project which consists of Arithmetic Logic Unit (ALU) and Register File with synchronized clock operations was succeeded in reading a number of instructions ,calculating their results and printing it , also compare the result whit the expected value of the same instructions .